API 一覧と詳細の履歴表示が 状態保持型エージェントの継続に与える影響

検討用原稿・著者情報未設定

2026 年 10 月 23 日

あらまし（検討用） 実行状態を保持した 4 課題 12 条件で API 一覧と詳細の履歴表示を比較した。詳細省略の既知総 token は全表示を上回り、使用量不明も 1 件生じた。保存状態と完了通知を区別し、開発比較の範囲を報告する。

原稿の位置づけ．既知課題での開発比較を一つの問いへ絞った検討用原稿である．研究上の中心的貢献，著者情報，公式様式への適合は未確定であり，投稿完成稿ではない．

1 目的と検証する問い

AI エージェントの費用を減らすには，必要な情報・処理と最終品質を守り，取得，生成，実行，検証を含む仕事全体を制御する必要がある．短い入力が終了までの費用を減らすとは限らず，圧縮後の行動や再取得，課題別の成否まで調べる必要がある [1]．本研究は，この長期的目標のうち，持続する実行状態と会話に見える情報との関係を扱う．

API を使う作業では，何ができるかを示す一覧と，具体的な引数を示す詳細の両方が現れる．詳細を読んだ後も長い説明を入力へ繰り返す負担がある一方，一覧まで省くと後続の行動選択を変える可能性がある．ただし，実行環境に情報が残る場合，表示から消えたことを情報の完全な喪失とは呼べない．

本稿の問いは，Python 状態と原文の参照経路を残した共通の途中状態で，API 一覧と詳細の表示を別々に省くと，最終成否と終了までの費用がどう変わるか，である．通常の段階的開示を新しいアルゴリズムとして提案するものではない．また，単一作業役の比較から，多担当間の通信削減を主張しない．

2 関連研究と比較範囲

Beyond Token Savings[1] は複数モデルと反復を用いて，圧縮方針と成否・token・時間の関係を検証する．Liu[2] は状態の再取得費用を分け，予算をそろえた保持や外部からの状態復元を比較する．環境によって再取得の増加が見られない例も報告している．従って，短縮が総費用削減を保証しないという一般論は，本稿独自の発見ではない．

CodeAct[3] はコードを行動として用いる．CaveAgent[4] は持続する実行状態を扱っており，Python 変数を保持すること自体は本研究の新規性ではない．HD-Gist[5] は API 引数と値を区別した学習表現を使い，必要な表現を生成中に動的に参照する．API 説明を種類に分けて圧縮する一般的着想も既存である．

Anthropic の Tool Search Tool[6] は，初期表示を抑えて検索したツールの定義を展開する．追加検索の時間と入力削減の釣合いも設計上の論点とされる．長文を扱う段階的開示の研究 [7] も，同じ文書・チャンクで索引の配置と階層を比較している．従って，索引を残すことや表示を分割することだけを独自の貢献としない．

今回の対象は，既に一度取得した API 説明の履歴表示である．操作先の環境と Python を復元し，公開スキーマと保存原文へのアクセスを共通にした上で，表示の違いと継続の成否・総費用を記録する．先行方式の実装を同一モデルで再現した比較ではないため，先行方式より優れるという主張は行わない．

3 状態と費用の定義

継続点の状態を会話履歴 H，Python 名前空間 P，操作先の環境 E，残予算 B とする．介入は H の一部に限り，P と E に対する操作権限をそろえる．API 名が過去コードに現れること，その意味を理解すること，正しい処理に使うことは別である．

仕事全体の論理トークンを

$$T_{\mathrm{work}} = T_{\mathrm{prefix}} + \sum_{j=1}^{n} (I_j + O_j) \tag{1}$$

とする．共通前半は保存した実測値を一度ずつ加え，キャッシュした入力も I_j へ含める．継続のみの値も併記する．API，時間，制御・計数，実験用復元，評価，推論器のロード・常駐を別記し，準備費用を無料とみなさない．評価器単独の秒数は実行器が保存しておらず未計

測とする．評価と親プロセスの処理は推論器の稼働全体に含まれ，継続・復元の測定値との差を評価器だけの時間へ読み替えない．

4　固定した開発比較

4.1　一覧と詳細を分けた表示比較

API 説明の省略による変化を調べるため，全表示 F，詳細のみ省略 D，同じ詳細と一覧を省略 B を設けた．古い成功出力のうち，保存した API 返却と表示処理から出自を照合できる区間だけを変更する．操作コード，リテラル，混合出力，エラー，最新応答を保持し，D と B で詳細の省略集合と省略印の形式をそろえる．Python 状態，公開スキーマ，原文の参照経路，公開契約の拒否時返信は共通である．従って，実行環境から知識を消去する介入ではない．

以前に観測済みの 4 課題の第 8 応答後を使い，各 3 条件を一度ずつ実行する計画を固定した．上限は各 16 生成，継続 120,000 token，480 秒とし，初回は入力キャッシュを再利用しない．全入力と残予算を計数して出力枠を制限する．独立評価器で保存環境の最終成否を判定し，仕事全体の論理トークンを $T_{\mathrm{prefix}}+\sum_j (I_j+O_j)$ とする．制御，計数，失敗，再取得を含め，復元・評価・推論器常駐は別記する．使用量不明の生成は実行も再試行もせず当該条件を終了する．API 作用と片付けが既知の場合のみ最終状態を評価するが，不明費用は補完しない．

推論には MiniMax-M3 の IQ4_XS 量子化版と llama.cpp b11669 を用いる．コンテキスト上限は 65,536，温度 1.0，top-p 0.95，出力上限 4,096 とした．継続の seed は 50 から呼出しごとに 1 増やし，条件間でそろえる．API 上限には前半の要求も含め，写真は 160，他の課題は 80 とする．順序は課題ごとに FDB，DBF，BFD，FBD で固定した．量子化版，推論器版，実装版，入力と返却のハッシュを保存し，実行条件を固定したことと結果の決定性を区別する．

主要な対照は D 対 F と D 対 B である．返金課題には追加で省ける一覧がなく，D/B の基礎入力が同じ負の対照となる．ここで生じた差は一覧保持の効果と解釈しない．既知 4 課題での開発比較のため，独立 12 課題や統計的な非劣性の証明とは扱わない．

4.2　対象と入力の照合

AppWorld[8] の既知 4 課題は写真整理，電気代精算，返金，曲一覧の書出である．過去の失敗と入力計数を見た後の方式探索であり，独立の確認評価ではない．未閲覧の 12 シナリオは使わず，有利な課題への交換もしない．元の返却，保存 DB，Python の出力を照合するが，プロセス全体のヒープや乱数状態を完全複製したとは主張しない．

D/B で詳細の省略対象は各 7・2・3・7 返却と一致した．B が追加で省く一覧は各 1・1・0・3 返却だった．初回入力は表 1 のとおりで，共通の予算通知と省略印を含む．この差を終了までの節約量へ転記しない．

表 1　通知を含む初回入力 token

課題	F	D	B
写真整理	8,789	7,342	6,848
電気代精算	11,267	11,009	10,202
返金	15,746	14,964	14,964
曲一覧の書出	13,771	12,142	9,473

4.3　採否と欠測

全 12 条件を評価し使用量が既知で監査に合格した場合に限り，D が F/B 双方の成功を失わず，合計 T_{work} が F より小さく B 以下なら，既知課題に対する採用候補とする．一覧の追加価値を支持するには，入力の異なる課題で D 成功・B 失敗の対も必要とする．4 課題の同数成功を統計的な非劣性とはみなさない．

使用量不明はゼロや上限で埋めず，既知分と不明生成数を示す．不明があれば確定した総費用差による採用判定は行わない．API 作用，復元，実行，片付け，評価，推論器版の不一致は全体を停止する．結果を見て seed・条件・予算を変更せず，未実行を成功や失敗に補完しない．

5　全予定条件の結果

全 12 予定条件のうち 12 件を評価し，0 件が未評価，0 件が未実行だった．継続費用は既知 1,000,847 token，使用量不明は 1 生成だった．詳細省略 D は固定の採用基準を満たさず，品質を保った総費用削減として採用しない．D は既知分だけで F を 43,083 token 上回るため，未知分を除いても削減にはならない．全 12 予定条件の成否と仕事全体の費用を表 2 へ示す．途中の入力が少なくても，生成回数や後続の行動が変われば合計は減らない．また，要求を満たさず早く停止した条件を，効率のよい成功と数えない．

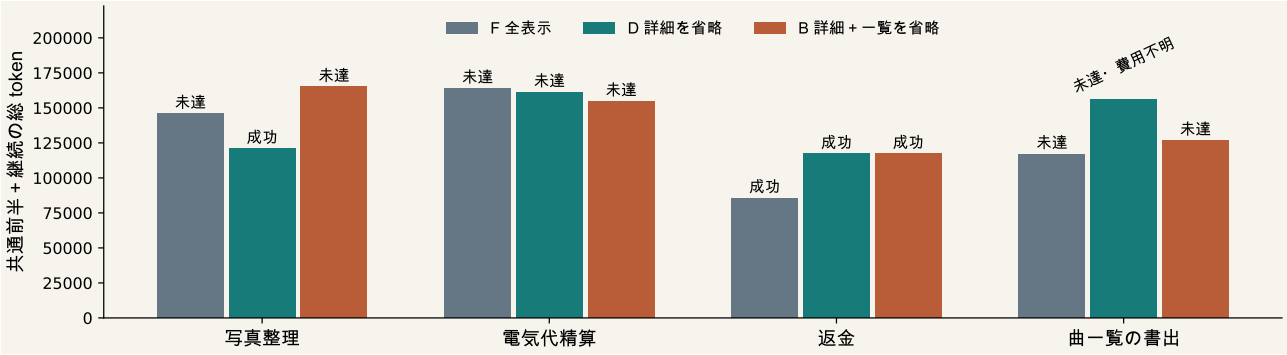


図 1　全 4 課題・3 表示の仕事全体 token と公式成否．共通前半を含み，縦軸は 0 から示す．斜線の棒は既知分のみで，さらに使用量不明の呼出しがある．

表 2　全 12 予定条件の成否と費用

課題	条件・成否	仕事全体 token	継続生成
写真整理	F・未達	145,850	8
	D・成功	120,935	6
	B・未達	165,015	11
電気代精算	F・未達	163,918	9
	D・未達	161,225	9
	B・未達	154,720	9
返金	F・成功	85,697	2
	D・成功	117,223	4
	B・成功	117,305	4
曲一覧の書出	F・未達	117,100	3
	D・未達	156,265+?	7
	B・未達	126,734	5

+? は別に使用量不明の 1 生成があることを示す．

5.1　費用差と実行量

全表示 F に対する全 4 課題の確定費用差は算出しない．一覧も省く B に対する全 4 課題の確定費用差は算出しない．表 3 は全条件の生成，API，継続時間を含む．継続 API は実際に送信した要求数であり，拒否され送信しなかった要求や原文参照の回数は別の記録に残す．API 回数だけを見て再取得や無駄を断定せず，完了できた処理と対にして解釈する．

表 3　全 4 課題合計の費用と実行量

指標	F	D	B
成功／評価済み	1/4	2/4	1/4
仕事全体・既知 token	512,565	555,648	563,774
継続・既知 token	302,185	345,268	353,394
使用量不明生成	0	1	0
前半＋継続生成	54	58	61
継続生成	22	26	29
前半＋継続 API	224	213	173
継続 API	173	162	122
復元 API	51	51	51
継続時間（秒）	1,308.5	1,354.0	1,359.1

5.2　課題別の変化をどう読むか

写真整理の F/D は，それぞれ 42 ／ 42 件のファイル移動要求を送ったが，成否は未達／成功だった．この対では全ての非 supervisor アプリの保存ファイルが一致し，異なるアプリ状態は supervisor だけだった．完了通知の要求は F で 0，D で 1 件であり，公式成否の差をファイル整理そのものの改善と呼ばない．隠れた個別テストの原因同定ではない．返金は F/D/B の全条件が成功した．継続 token は F/D で 32,209 ／ 63,735 であり，初回の詳細省略量だけから終了までの費用を予測できない．要求を送った回数は，処理内容や要求達成の同一性を保証しない．仕事の終了条件をそろえた上で，成功と費用を対にして解釈する．

5.3　表示から消えた情報の範囲

一覧を省いた B でも，写真の一覧 26 API 名のうち 8 名，電気代では 26 名のうち 10 名が入力中に文字列として残った．過去に実行したコードや詳細取得の操作にも API 名が含まれるためである．書出の 3 一覧も 9 ／ 91，2 ／ 26，6 ／ 6 名が残った．同名 API による偶然の一致を含み得るため，これは意味理解の測定ではない．一覧を消したことと，API の存在を全く知らないことを混同しない．

返金では D/B の基礎入力が同じだった．異なる結果が生じた場合も一覧の効果には帰属させない．残時間通知，実行時間，数値計算などの変動が残る．過去 P47 と今回 P55 の全表示条件について，両方を評価できた比較のうち 1 課題で成否が異なった．これは事前に反復数を割り付けた追試ではなく，変更の原因や分散の推定には用いない．

6　主張の限界と次の確認

本比較は，既知 4 課題，単一モデル，各条件 1 回に限られる．一つの課題で成功と費用が改善しても，広い課題群の品質維持や因果機構の確立とはならない．同じ seed と初回キャッシュ不使用は推論条件をそろえる措置だが，経路全体の再現性を保証しない．後続の入力と処理の変化を，独立した削減効果の和に分解することもし

ない．

表示対象は出自が照合できた古い説明に限定され，混合出力，最新応答，不確かな表示を残す．従って，任意の会話や API 説明へ同じ効果を保証しない．結果が有望でも，機構・比較・予算・分析を固定した未調整群と，生成変動の確認が必要である．新しいデータや限定した解釈を貢献の候補とする場合にも，既存の段階的開示に対する追加の知見が何かを明示しなければならない．

7　現在の結論

状態保持型エージェントについて，API 一覧と詳細の履歴表示を分けた開発比較を行った．保存原文と実行状態を共通にし，最終成否，仕事全体の費用，欠測，実行量を全予定条件で記録した．この局所比較を，品質を保ちながら不要な取得・共有・生成・実行・検証を減らす研究へつなぐ．通常方式の実装や少数例の改善を，新しい万能な制御方式の確立へ読み替えない．

参考文献

[1] R. Satish et al., “Beyond Token Savings: A Systematic Study of Context Compression in LLM Agents,” arXiv:2609.32961v1, 2026.

[2] S. Liu, “What Does Context Compression Cost an Agent? Interaction Costs Unrevealed by Task-Completion Metrics,” arXiv:2608.16370v1, 2026.

[3] X. Wang et al., “Executable Code Actions Elicit Better LLM Agents,” ICML, PMLR 235, pp.50208–50232, 2024.

[4] M. Ran et al., “CaveAgent: Transforming LLMs into Stateful Runtime Operators,” arXiv:2601.01569v4, 2026.

[5] Y. Jiang, M. Del Vecchio, M. Bansal, and A. Johannsen, “Hierarchical and Dynamic Prompt Compression for Efficient Zero-shot API Usage,” Findings of EACL, pp.2162–2174, 2024.

[6] Anthropic, “Introducing advanced tool use on the Claude Developer Platform,” Nov. 24, 2025.

[7] Y. He, Y. Zhao, J. Wang, and H. Chen, “Is Progressive Disclosure All You Need for Long-Context Agents?,” arXiv:2607.17598v1, 2026.

[8] H. Trivedi et al., “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents,” ACL, pp.16022–16076, 2024.